

```

import React, { useState, useEffect } from 'react';
import { LineChart, Line, ScatterChart, Scatter, XAxis, YAxis, CartesianGrid, Too

const DSRRiemannTest = () => {
  const [results, setResults] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    runAnalysis();
  }, []);

  const runAnalysis = () => {
    // Riemann zeros imaginary parts (from uploaded file)
    const imParts = [
      14.1347251417, 21.0220396388, 25.0108575801, 30.4248761259, 32.9350615877,
      37.5861781588, 40.9187190121, 43.3270732809, 48.0051508812, 49.7738324777,
      52.9703214777, 56.4462476971, 59.3470440026, 60.8317785246, 65.1125440481,
      67.0798105295, 69.5464017112, 72.0671576745, 75.7046906991, 77.1448400689,
      79.3373750202, 82.9103808541, 84.7354929805, 87.4252746131, 88.8091112076,
      92.4918992706, 94.6513440405, 95.8706342282, 98.8311942182, 101.3178510057,
      103.7255380405, 105.4466230523, 107.1686111843, 111.0295355432, 111.8746591
      114.3202209155, 116.2266803209, 118.7907828660, 121.3701250024, 122.9468292
      124.2568185543, 127.5166838796, 129.5787042000, 131.0876885309, 133.4977372
      134.7565097534, 138.1160420545, 139.7362089521, 141.1237074040, 143.1118458
      146.0009824868, 147.4227653426, 150.0535204208, 150.9252576122, 153.0246938
      156.1129092942, 157.5975918176, 158.8499881714, 161.1889641376, 163.0307096
      165.5370691879, 167.1844399782, 169.0945154156, 169.9119764794, 173.4115365
      174.7541915234, 176.4414342977, 178.3774077761, 179.9164840203, 182.2070784
      184.8744678484, 185.5987836777, 187.2289225835, 189.4161586560, 192.0266563
      193.0797266038, 195.2653966795, 196.8764818410, 198.0153096763, 201.2647519
      202.4935945141, 204.1896718031, 205.3946972022, 207.9062588878, 209.5765097
      211.6908625954, 213.3479193597, 214.5470447835, 216.1695385083, 219.0675963
      220.7149188393, 221.4307055547, 224.0070002546, 224.9833246696, 227.4214442
      229.3374133055, 231.2501887005, 231.9872352532, 233.6934041789, 236.5242296
    ];

    // Under RH, all zeros have Re = 0.5, so x = Re - 0.5 = 0
    // This is the critical test: do zeros show ANY Re-deviation?

    // For this test, we'll assume theoretical Re = 0.5 exactly
    // and check if there's any phase structure in the Im distribution

    const y = imParts;
    const x = new Array(y.length).fill(0); // Under RH: Re(ρ) = 0.5 exactly
  }
}

```

```

// Training window: Im  $\in$  [14.13, 60]
const trainMask = y.map(val => val >= 14.13 && val <= 60);
const xTrain = x.filter((_, i) => trainMask[i]);
const yTrain = y.filter((_, i) => trainMask[i]);

console.log(`Training on ${yTrain.length} zeros in Im  $\in$  [14.13, 60]`);

// Since  $x \approx 0$  for all zeros (RH assumption), we need to test differently
// Instead, let's check the SPACING between consecutive zeros
const spacings = [];
for (let i = 1; i < y.length; i++) {
  spacings.push({
    y: (y[i] + y[i-1]) / 2,
    spacing: y[i] - y[i-1]
  });
}

// Test if spacing shows DSR periodicity
const spacingTrain = spacings.filter(s => s.y >= 14.13 && s.y <= 60);

// Fit: spacing = mean + A*cos(k*y +  $\phi$ )
const meanSpacing = spacingTrain.reduce((sum, s) => sum + s.spacing, 0) / spa

// Grid search for k, A,  $\phi$ 
let bestK = 0, bestA = 0, bestPhi = 0, bestRMS = Infinity;

for (let k = 0.1; k <= 1.5; k += 0.05) {
  for (let A = 0.1; A <= 2.0; A += 0.1) {
    for (let phi = -Math.PI; phi <= Math.PI; phi += Math.PI/8) {
      let sumSq = 0;
      spacingTrain.forEach(s => {
        const predicted = meanSpacing + A * Math.cos(k * s.y + phi);
        const error = s.spacing - predicted;
        sumSq += error * error;
      });
      const rms = Math.sqrt(sumSq / spacingTrain.length);

      if (rms < bestRMS) {
        bestRMS = rms;
        bestK = k;
        bestA = A;
        bestPhi = phi;
      }
    }
  }
}

```

```

// Generate predictions
const spacingTest = spacings.filter(s =>
  (s.y >= 69 && s.y <= 73) ||
  (s.y >= 79 && s.y <= 83) ||
  (s.y >= 99 && s.y <= 103)
);

const predictions = spacingTest.map(s => ({
  y: s.y,
  actual: s.spacing,
  predicted: meanSpacing + bestA * Math.cos(bestK * s.y + bestPhi),
  error: Math.abs(s.spacing - (meanSpacing + bestA * Math.cos(bestK * s.y + b
})));

// Create visualization data
const allSpacingData = spacings.map(s => ({
  y: s.y,
  actual: s.spacing,
  theory: meanSpacing + bestA * Math.cos(bestK * s.y + bestPhi),
  inTraining: s.y >= 14.13 && s.y <= 60
})));

setResults({
  params: {
    k: bestK.toFixed(4),
    A: bestA.toFixed(4),
    phi: bestPhi.toFixed(4),
    meanSpacing: meanSpacing.toFixed(4),
    rms: bestRMS.toFixed(4)
  },
  training: {
    nZeros: yTrain.length,
    nSpacings: spacingTrain.length,
    imRange: `[${yTrain[0].toFixed(2)}, ${yTrain[yTrain.length-1].toFixed(2)}]`,
  },
  predictions,
  visualData: allSpacingData.slice(0, 80) // First 80 for visualization
});

setLoading(false);
};

if (loading) {
  return <div className="flex items-center justify-center h-screen">
    <div className="text-xl">Running DSR Test 1 on Riemann Zeros...</div>
  </div>;
}

```

```
}
```

```
const avgError = results.predictions.reduce((sum, p) => sum + p.error, 0) / res
```

```
return (
```

```
<div className="w-full h-screen overflow-auto bg-gray-50 p-8">
```

```
<div className="max-w-7xl mx-auto">
```

```
<h1 className="text-3xl font-bold mb-6 text-gray-900">
```

```
  DSR Test 1: Riemann Zero Spacing Analysis
```

```
</h1>
```

```
<div className="bg-white rounded-lg shadow-lg p-6 mb-6">
```

```
<h2 className="text-xl font-semibold mb-4 text-gray-800">Fitted Paramet
```

```
<div className="grid grid-cols-2 md:grid-cols-3 gap-4">
```

```
<div className="bg-blue-50 p-4 rounded">
```

```
<div className="text-sm text-gray-600">Frequency (k)</div>
```

```
<div className="text-2xl font-bold text-blue-600">{results.params.k
```

```
</div>
```

```
<div className="bg-green-50 p-4 rounded">
```

```
<div className="text-sm text-gray-600">Amplitude (A)</div>
```

```
<div className="text-2xl font-bold text-green-600">{results.params.
```

```
</div>
```

```
<div className="bg-purple-50 p-4 rounded">
```

```
<div className="text-sm text-gray-600">Phase ( $\phi$ )</div>
```

```
<div className="text-2xl font-bold text-purple-600">{results.params
```

```
</div>
```

```
<div className="bg-orange-50 p-4 rounded">
```

```
<div className="text-sm text-gray-600">Mean Spacing</div>
```

```
<div className="text-2xl font-bold text-orange-600">{results.params
```

```
</div>
```

```
<div className="bg-red-50 p-4 rounded">
```

```
<div className="text-sm text-gray-600">Training RMS</div>
```

```
<div className="text-2xl font-bold text-red-600">{results.params.rm
```

```
</div>
```

```
<div className="bg-indigo-50 p-4 rounded">
```

```
<div className="text-sm text-gray-600">Forecast Error</div>
```

```
<div className="text-2xl font-bold text-indigo-600">{avgError.toFixed
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className="bg-white rounded-lg shadow-lg p-6 mb-6">
```

```
<h2 className="text-xl font-semibold mb-4 text-gray-800">Training Data<
```

```
<div className="grid grid-cols-3 gap-4 text-sm">
```

```
<div>
```

```
<span className="font-semibold">Zeros in training:</span> {results.
```

```
</div>
```

```

    <div>
      <span className="font-semibold">Spacing samples:</span> {results.tr
    </div>
    <div>
      <span className="font-semibold">Im range:</span> {results.training.
    </div>
  </div>
</div>

```

```

<div className="bg-white rounded-lg shadow-lg p-6 mb-6">
  <h2 className="text-xl font-semibold mb-4 text-gray-800">Zero Spacing v
  <ResponsiveContainer width="100%" height={400}>
    <LineChart data={results.visualData}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis
        dataKey="y"
        label={{ value: 'Im( $\rho$ )', position: 'insideBottom', offset: -5 }}
      />
      <YAxis
        label={{ value: 'Spacing between zeros', angle: -90, position: 'i
      />
      <Tooltip />
      <Legend />
      <Line
        type="monotone"
        dataKey="theory"
        stroke="#3b82f6"
        strokeWidth={2}
        dot={false}
        name="DSR Model"
      />
      <Line
        type="monotone"
        dataKey="actual"
        stroke="#000000"
        strokeWidth={1}
        dot={{ r: 2 }}
        name="Actual Spacing"
      />
    </LineChart>
  </ResponsiveContainer>
</div>

```

```

<div className="bg-white rounded-lg shadow-lg p-6">
  <h2 className="text-xl font-semibold mb-4 text-gray-800">Forecast Resul
  <div className="overflow-x-auto">
    <table className="w-full text-sm">

```

```

    <thead className="bg-gray-100">
      <tr>
        <th className="p-3 text-left">Im( $\rho$ )</th>
        <th className="p-3 text-left">Actual Spacing</th>
        <th className="p-3 text-left">Predicted Spacing</th>
        <th className="p-3 text-left">Absolute Error</th>
      </tr>
    </thead>
    <tbody>
      {results.predictions.map((p, i) => (
        <tr key={i} className={i % 2 === 0 ? 'bg-gray-50' : ''}>
          <td className="p-3">{p.y.toFixed(2)}</td>
          <td className="p-3">{p.actual.toFixed(4)}</td>
          <td className="p-3">{p.predicted.toFixed(4)}</td>
          <td className="p-3 font-semibold">{p.error.toFixed(4)}</td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
</div>

<div className="bg-yellow-50 border-l-4 border-yellow-400 p-6 mt-6">
  <h3 className="font-semibold text-yellow-900 mb-2">Analysis Notes</h3>
  <p className="text-sm text-yellow-800">
    Under the Riemann Hypothesis, all non-trivial zeros have  $\text{Re}(\rho) = 0.5$ .
    Therefore, Test 1 examines whether the <strong>spacing pattern</strong>
    zeros exhibits DSR periodicity as a function of  $\text{Im}(\rho)$ . The model fit:
     $\text{spacing}(y) = \text{mean} + A \cdot \cos(k \cdot y + \phi)$  where  $y = \text{Im}(\rho)$ .
  </p>
  <p className="text-sm text-yellow-800 mt-2">
    <strong>Key question:</strong> Does  $k \approx 0.63$  emerge from real zero data?
  </p>
</div>
</div>
);
};

export default DSR_RiemannTest;

```